```python
#Task 1: Data Preparation
#importing the required libraries to perform operations like data cleaning, data preparation, data analysis and visualisation.
import pandas as pd #Pandas is a Python library used for data analysis and data manipulation.
import numpy as np#NumPy (Numerical Python) is a fundamental Python library for numerical and scientific computing.
import matplotlib.pyplot as plt # Python library used to create graphs, charts, and visualizations.
import seaborn as sns #Python data visualization library built on top of matplotlib.

#1.Reading the csv file
df=pd.read_csv('Retail_Transactions_Dataset.csv')

df
```

	Transaction_ID	Date	Customer_Name	Product	Total_Items
0	1000000000	2022-01-21 06:27:29	Stacey Price	['Ketchup', 'Shaving Cream', 'Light Bulbs']	3
1	1000000001	2023-03-01 13:01:21	Michelle Carlson	['Ice Cream', 'Milk', 'Olive Oil', 'Bread', 'P...	2
2	1000000002	2024-03-21 15:37:04	Lisa Graves	['Spinach']	6
3	1000000003	2020-10-31 09:59:47	Mrs. Patricia May	['Tissues', 'Mustard']	1
4	1000000004	2020-12-10 00:59:59	Susan Mitchell	['Dish Soap']	10
...	...	...	...	...	...
999995	1000999995	2023-03-27 06:12:10	Lisa Gonzalez	['Pickles', 'Carrots', 'Peanut Butter', 'Spong...	1
999996	1000999996	2022-05-19 05:13:58	Emily Graham	['Cereal']	8
999997	1000999997	2021-09-03 13:59:39	Cynthia Anderson	['Trash Bags']	3
999998	1000999998	2023-10-17 05:50:40	Michael Rodriguez	['Diapers', 'Coffee', 'Coffee', 'Mop']	3
999999	1000999999	2020-06-15 11:58:49	Jennifer Davis		

```
999999                       ['Trash Cans', 'Mop', 'Jam']            8


        Total_Cost  Payment_Method           City
Store_Type  \
0            71.65  Mobile Payment    Los Angeles     Warehouse Club

1            25.93            Cash  San Francisco    Specialty Store

2            41.49     Credit Card        Houston   Department Store

3            39.34  Mobile Payment        Chicago           Pharmacy

4            16.42      Debit Card        Houston    Specialty Store

...            ...             ...            ...                ...

999995       22.07      Debit Card    Los Angeles        Supermarket

999996       80.25            Cash        Houston     Warehouse Club

999997       60.74     Credit Card    Los Angeles  Convenience Store

999998       23.48      Debit Card  San Francisco        Supermarket

999999       44.12     Credit Card        Atlanta           Pharmacy


        Discount_Applied Customer_Category  Season
Promotion
0                   True         Homemaker  Winter
NaN
1                   True      Professional    Fall      BOGO (Buy One
Get One)
2                   True      Professional  Winter
NaN
3                   True         Homemaker  Spring
NaN
4                  False       Young Adult  Winter  Discount on
Selected Items
...                  ...               ...     ...
...
999995             False       Middle-Aged  Winter
NaN
999996              True    Senior Citizen  Spring  Discount on
Selected Items
999997             False         Homemaker  Winter
NaN
999998              True           Retiree  Winter      BOGO (Buy One
Get One)
```

```
999999             False      Professional    Fall  Discount on
Selected Items

[1000000 rows x 13 columns]
```

```
#4.Preprocessing the data by checking if there are any null values and
filling it with a default value
df['Promotion'].isna().sum()
```

```
np.int64(333943)
```

```
df["Promotion"] = df["Promotion"].fillna("No Promotion")
```

```
df
```

```
        Transaction_ID                 Date      Customer_Name  \
0           1000000000  2022-01-21 06:27:29       Stacey Price
1           1000000001  2023-03-01 13:01:21   Michelle Carlson
2           1000000002  2024-03-21 15:37:04        Lisa Graves
3           1000000003  2020-10-31 09:59:47  Mrs. Patricia May
4           1000000004  2020-12-10 00:59:59     Susan Mitchell
...                ...                  ...                ...
999995      1000999995  2023-03-27 06:12:10      Lisa Gonzalez
999996      1000999996  2022-05-19 05:13:58       Emily Graham
999997      1000999997  2021-09-03 13:59:39   Cynthia Anderson
999998      1000999998  2023-10-17 05:50:40  Michael Rodriguez
999999      1000999999  2020-06-15 11:58:49     Jennifer Davis

                                                 Product  Total_Items
\
0             ['Ketchup', 'Shaving Cream', 'Light Bulbs']            3

1       ['Ice Cream', 'Milk', 'Olive Oil', 'Bread', 'P...            2

2                                           ['Spinach']            6

3                                  ['Tissues', 'Mustard']            1

4                                          ['Dish Soap']           10

...                                                  ...          ...

999995  ['Pickles', 'Carrots', 'Peanut Butter', 'Spong...            1

999996                                         ['Cereal']            8

999997                                     ['Trash Bags']            3

999998             ['Diapers', 'Coffee', 'Coffee', 'Mop']            3

999999                         ['Trash Cans', 'Mop', 'Jam']            8
```

```
        Total_Cost  Payment_Method           City
Store_Type  \
0            71.65  Mobile Payment    Los Angeles     Warehouse Club

1            25.93            Cash  San Francisco    Specialty Store

2            41.49     Credit Card        Houston   Department Store

3            39.34  Mobile Payment        Chicago           Pharmacy

4            16.42      Debit Card        Houston    Specialty Store

...            ...             ...            ...                ...

999995       22.07      Debit Card    Los Angeles        Supermarket

999996       80.25            Cash        Houston     Warehouse Club

999997       60.74     Credit Card    Los Angeles  Convenience Store

999998       23.48      Debit Card  San Francisco        Supermarket

999999       44.12     Credit Card        Atlanta           Pharmacy


        Discount_Applied Customer_Category  Season
Promotion
0                   True         Homemaker  Winter                No
Promotion
1                   True      Professional    Fall      BOGO (Buy One
Get One)
2                   True      Professional  Winter                No
Promotion
3                   True         Homemaker  Spring                No
Promotion
4                  False       Young Adult  Winter  Discount on
Selected Items
...                  ...               ...     ...
...
999995             False       Middle-Aged  Winter                No
Promotion
999996              True    Senior Citizen  Spring  Discount on
Selected Items
999997             False         Homemaker  Winter                No
Promotion
999998              True           Retiree  Winter      BOGO (Buy One
Get One)
999999             False      Professional    Fall  Discount on
Selected Items
```

```
[1000000 rows x 13 columns]
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 13 columns):
 #   Column             Non-Null Count    Dtype  
---  ------             --------------    -----  
 0   Transaction_ID     1000000 non-null  int64  
 1   Date               1000000 non-null  object 
 2   Customer_Name      1000000 non-null  object 
 3   Product            1000000 non-null  object 
 4   Total_Items        1000000 non-null  int64  
 5   Total_Cost         1000000 non-null  float64
 6   Payment_Method     1000000 non-null  object 
 7   City               1000000 non-null  object 
 8   Store_Type         1000000 non-null  object 
 9   Discount_Applied   1000000 non-null  bool   
 10  Customer_Category  1000000 non-null  object 
 11  Season             1000000 non-null  object 
 12  Promotion          1000000 non-null  object 
dtypes: bool(1), float64(1), int64(2), object(9)
memory usage: 92.5+ MB
```

```
#2.Parse and convert the Date column into an appropriate format.
#here the 'Date' column is of object data type and we cannot perform
any date operations on it, so we need to convert it into datatime data
type
df['Date']=pd.to_datetime(df['Date'])
```

```
#Extract additional useful information like Year, Month, or DayOfWeek
from the Date.
#using the builtin functions of datetime to extract year and month
from Date.
df['year']=df['Date'].dt.year
```

```
df['Month']=df['Date'].dt.month
df['DayOfWeek']=df['Date'].dt.day_name()
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 16 columns):
 #   Column             Non-Null Count    Dtype         
---  ------             --------------    -----         
 0   Transaction_ID     1000000 non-null  int64         
 1   Date               1000000 non-null  datetime64[ns]
 2   Customer_Name      1000000 non-null  object 
```

```
 3   Product            1000000 non-null  object
 4   Total_Items        1000000 non-null  int64
 5   Total_Cost         1000000 non-null  float64
 6   Payment_Method     1000000 non-null  object
 7   City               1000000 non-null  object
 8   Store_Type         1000000 non-null  object
 9   Discount_Applied   1000000 non-null  bool
 10  Customer_Category  1000000 non-null  object
 11  Season             1000000 non-null  object
 12  Promotion          1000000 non-null  object
 13  year               1000000 non-null  int32
 14  Month              1000000 non-null  int32
 15  DayOfWeek          1000000 non-null  object
dtypes: bool(1), datetime64[ns](1), float64(1), int32(2), int64(2),
object(9)
memory usage: 107.8+ MB
```

```
#Task 2: Basic exploration

# 1.How many total transactions are there?
df.shape[0] # shape of the data will give the total transactions
```

```
1000000
```

```
# 2.How many unique customers are in the dataset?
print(df['Customer_Name'].nunique()) #nunique function will give the
count of unique values in the mentioned column
```

```
329738
```

```
# 3.What are the top 5 most common products sold across all
transactions?
top5_products=df['Product'].value_counts().head(5) #value)counts() is
used to find how many times each unique value appears in a column
top5_products
```

```
Product
['Toothpaste']    4893
['Deodorant']     2541
['Honey']         2540
['Eggs']          2515
['Vinegar']       2505
Name: count, dtype: int64
```

```
# 4.Which cities have the highest number of transactions?
print(df['City'].value_counts())
```

```
City
Boston           100566
Dallas           100559
```

```
Seattle          100167
Chicago          100059
Houston          100050
New York         100007
Los Angeles       99879
Miami             99839
San Francisco     99808
Atlanta           99066
Name: count, dtype: int64
```

```
#Task 3: Customer Behavior Analysis

# 1.Which customer categories spend the most on average?
#we need to find the average of total cost of each customer category,
this will be done only using group by
df.groupby('Customer_Category')
['Total_Cost'].mean().sort_values(ascending=False)
```

```
Customer_Category
Teenager          52.529091
Professional      52.525762
Student           52.487994
Homemaker         52.461417
Young Adult       52.448246
Retiree           52.435589
Middle-Aged       52.411318
Senior Citizen    52.342672
Name: Total_Cost, dtype: float64
```

```
# 2.Do certain customer categories prefer specific payment methods?
# crosstab (cross-tabulation) is a pandas function used to create a
table that shows the frequency/count of combinations of two
categorical columns.
#crosstabulation is used to find the frequency of one column each
unique category correspondance to frequency of another column each
unique category.
pd.crosstab(df['Customer_Category'],df['Payment_Method'])
```

Payment_Method	Cash	Credit Card	Debit Card	Mobile Payment
Customer_Category				
Homemaker	31360	31413	31315	31330
Middle-Aged	31078	31049	31198	31311
Professional	31246	31201	31118	31086
Retiree	31269	31408	31051	31344
Senior Citizen	31442	31425	31420	31198
Student	31307	31058	31287	31190
Teenager	31203	31467	31322	31327
Young Adult	31325	30964	31363	30925

```python
# 3.What is the average number of items bought per transaction per store type?
# we need to use groupby to find the avg items bought per transaction w.r.t store type
df.groupby('Store_Type')['Total_Items'].mean().sort_values(ascending=False)
```

```
Store_Type
Specialty Store      5.508395
Convenience Store    5.505574
Pharmacy             5.498182
Department Store     5.495547
Supermarket          5.485767
Warehouse Club       5.482233
Name: Total_Items, dtype: float64
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000000 entries, 0 to 999999
Data columns (total 16 columns):
 #   Column             Non-Null Count    Dtype         
---  ------             --------------    -----         
 0   Transaction_ID     1000000 non-null  int64         
 1   Date               1000000 non-null  datetime64[ns]
 2   Customer_Name      1000000 non-null  object        
 3   Product            1000000 non-null  object        
 4   Total_Items        1000000 non-null  int64         
 5   Total_Cost         1000000 non-null  float64       
 6   Payment_Method     1000000 non-null  object        
 7   City               1000000 non-null  object        
 8   Store_Type         1000000 non-null  object        
 9   Discount_Applied   1000000 non-null  bool          
 10  Customer_Category  1000000 non-null  object        
 11  Season             1000000 non-null  object        
 12  Promotion          1000000 non-null  object        
 13  year               1000000 non-null  int32         
 14  Month              1000000 non-null  int32         
 15  DayOfWeek          1000000 non-null  object        
dtypes: bool(1), datetime64[ns](1), float64(1), int32(2), int64(2), object(9)
memory usage: 107.8+ MB
```

```python
#Task 4. Promption and Discount Impact

# 1.What is the average cost of transactions where a discount was applied vs not applied?
#df['Discount_Applied'].unique()

df.groupby('Discount_Applied')['Total_Cost'].mean()
```

```
Discount_Applied
False    52.423512
True     52.486915
Name: Total_Cost, dtype: float64
```

```
# 2.Compare the average number of items purchased for different
promotion types.
df.groupby('Promotion')['Total_Items'].mean()
```

```
Promotion
BOGO (Buy One Get One)        5.494351
Discount on Selected Items    5.501248
No Promotion                  5.492228
Name: Total_Items, dtype: float64
```

```
# 3.Which promotion type seems to be most effective in terms of
increasing total cost?
df.groupby('Promotion')
['Total_Cost'].mean().sort_values(ascending=False)
#the promotion type with highest average total cost is consideed most
effective because it leads to higher customer spending on average.
```

```
Promotion
No Promotion                  52.565973
BOGO (Buy One Get One)        52.418500
Discount on Selected Items    52.380922
Name: Total_Cost, dtype: float64
```

```
#Task 5: Seasonality Trends
```

```
# 1.Which season has the highest total revenue?
df.groupby('Season')['Total_Cost'].sum().sort_values(ascending=False)
```

```
Season
Fall      13136913.71
Summer    13116675.79
Spring    13113238.75
Winter    13088392.15
Name: Total_Cost, dtype: float64
```

```
# 2.Are there seasonal preferences for certain store types or product
categories?
pd.crosstab(df['Season'],df['Store_Type'])
```

```
Store_Type  Convenience Store  Department Store  Pharmacy  Specialty
Store  \
Season

Fall                    41654             41593     41852
41606
```

```
Spring                  41798             41590     41825
41570
Summer                  41735             41623     41520
41362
Winter                  41562             41808     41718
41563
```

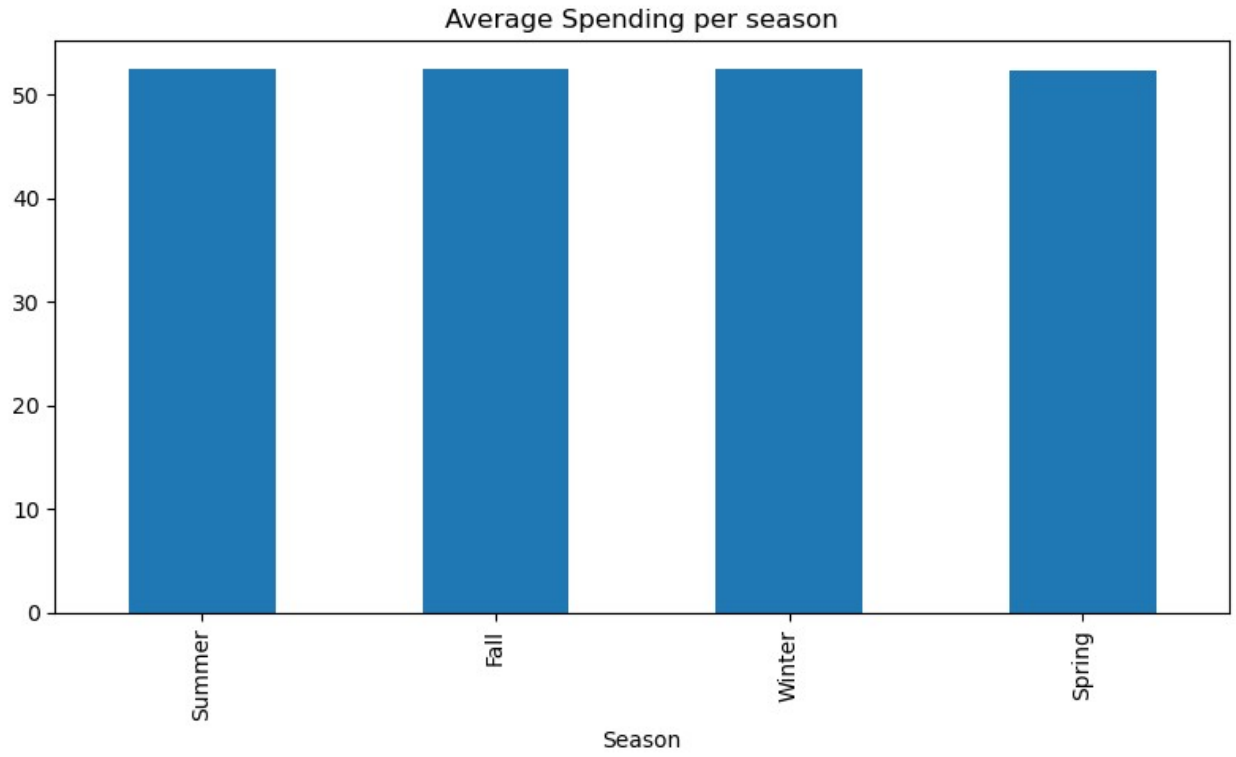
```
Store_Type  Supermarket  Warehouse Club
Season
Fall               41526           42017
Spring             41845           41740
Summer             41953           41428
Winter             41612           41500
```

```
# 3.Create a plot showing average spending per season.
avg_spending=df.groupby('Season')
['Total_Cost'].mean().sort_values(ascending=False) #finding the
average spending per season

avg_spending
```

```
Season
Summer    52.546363
Fall      52.495579
Winter    52.403247
Spring    52.375858
Name: Total_Cost, dtype: float64
```

```
plt.figure(figsize=(8,5))
avg_spending.plot(kind="bar")
plt.xlabel="Season"
plt.ylabel="Average Spending"
plt.title("Average Spending per season")
plt.tight_layout()
plt.show()
```



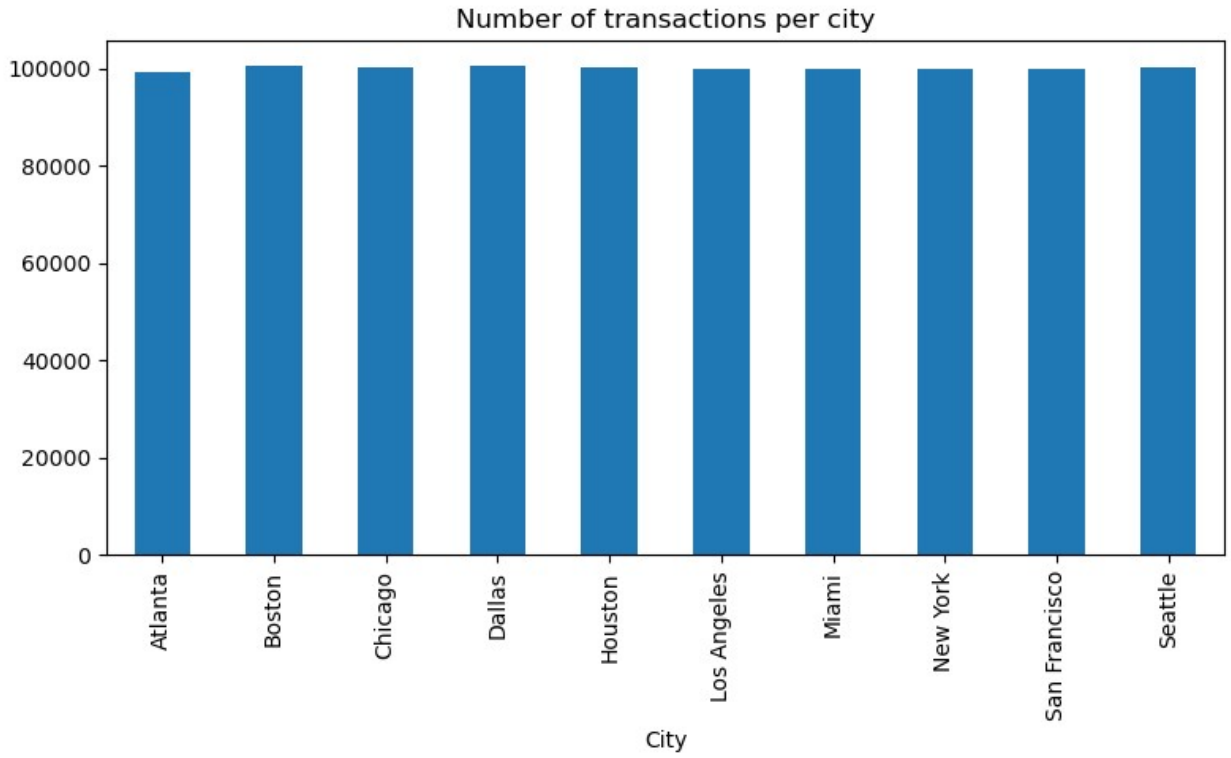

```
#Task 6: Visualization Dashboard

# 1.Bar plot of the number of transactions per city
no_of_transactions=df.groupby('City')['Transaction_ID'].count()
no_of_transactions
```

```
City
Atlanta          99066
Boston          100566
Chicago         100059
Dallas          100559
Houston         100050
Los Angeles      99879
Miami            99839
New York        100007
San Francisco    99808
Seattle         100167
Name: Transaction_ID, dtype: int64
```

```
plt.figure(figsize=(8,5))
no_of_transactions.plot(kind="bar")
plt.xlabel="City"
plt.ylabel="No.of.transactions"
plt.title("Number of transactions per city")
```

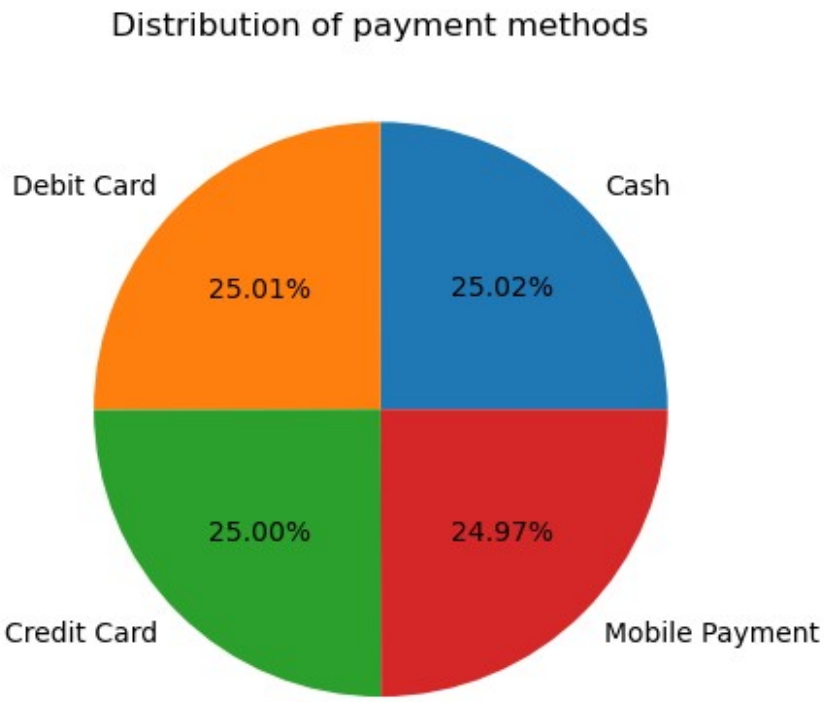
```
plt.tight_layout()
plt.show()
```




```
# 2.Pie chart showing distribution of payment methods
payment_counts=df['Payment_Method'].value_counts()
payment_counts
```

```
Payment_Method
Cash              250230
Debit Card        250074
Credit Card       249985
Mobile Payment    249711
Name: count, dtype: int64
```

```
plt.pie(payment_counts, labels=payment_counts.index, autopct='%1.2f%
%')
plt.title("Distribution of payment methods")
plt.show()
```




```
# 3.Line chart of monthly revenue trends (grouped by year if
applicable)

monthly_revenue = df.groupby(["year", "Month"])
["Total_Cost"].sum().reset_index()
monthly_revenue
```

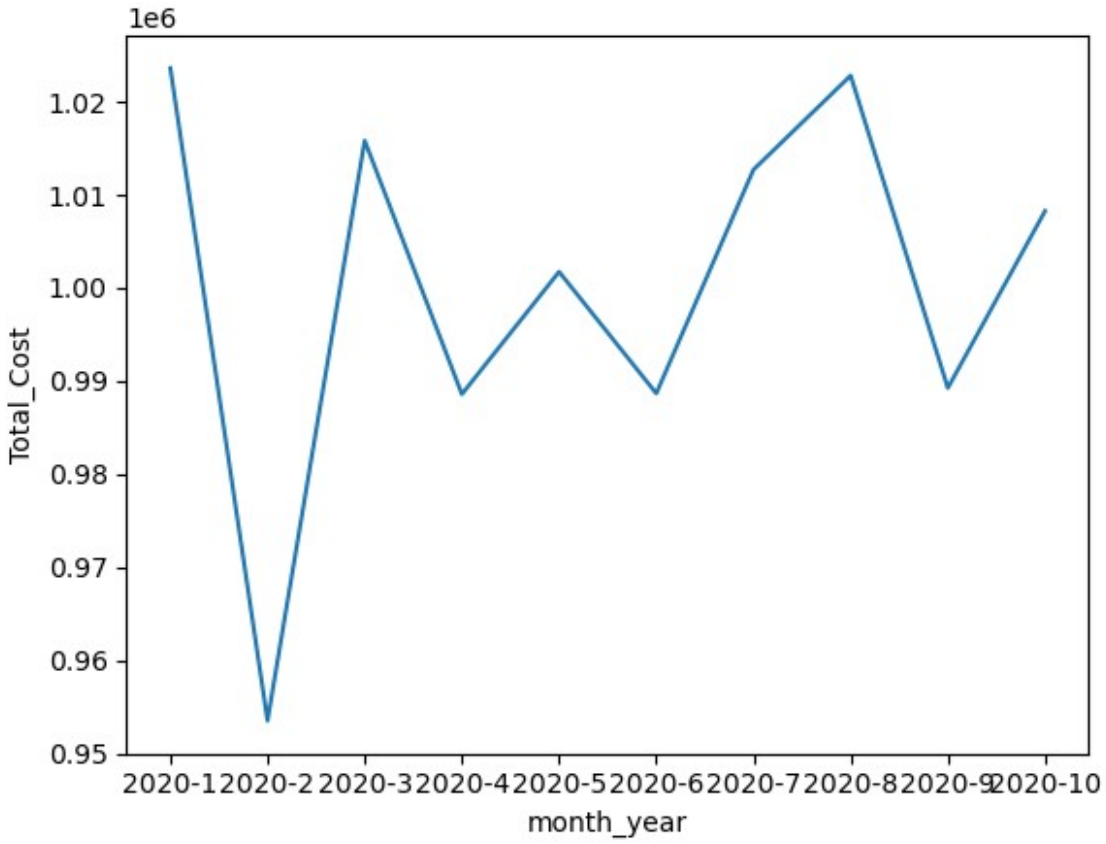
	year	Month	Total_Cost
0	2020	1	1023565.45
1	2020	2	953488.19
2	2020	3	1015781.37
3	2020	4	988549.87
4	2020	5	1001696.33
5	2020	6	988630.18
6	2020	7	1012661.97
7	2020	8	1022764.43
8	2020	9	989218.28
9	2020	10	1008214.78
10	2020	11	998095.24
11	2020	12	1012884.46
12	2021	1	1019424.75
13	2021	2	914130.51
14	2021	3	1021530.14
15	2021	4	977534.57
16	2021	5	1011122.43

```
17  2021      6   991433.74
18  2021      7  1021536.65
19  2021      8  1010434.45
20  2021      9   971126.82
21  2021     10  1023812.22
22  2021     11   992148.55
23  2021     12  1014676.09
24  2022      1  1019373.51
25  2022      2   910431.32
26  2022      3  1016946.28
27  2022      4   988888.00
28  2022      5  1020629.00
29  2022      6   996858.89
30  2022      7  1016872.19
31  2022      8  1006269.54
32  2022      9   977087.51
33  2022     10  1008172.01
34  2022     11   983624.09
35  2022     12  1001880.56
36  2023      1  1008107.16
37  2023      2   926532.97
38  2023      3  1033556.59
39  2023      4   978488.79
40  2023      5  1014930.61
41  2023      6   965819.82
42  2023      7  1029246.18
43  2023      8  1015234.15
44  2023      9   985031.04
45  2023     10  1019665.03
46  2023     11   989565.16
47  2023     12  1017350.68
48  2024      1  1013783.66
49  2024      2   943285.44
50  2024      3  1019427.01
51  2024      4   976736.25
52  2024      5   586965.49
```

```
monthly_revenue['month_year']=monthly_revenue['year'].astype(str)
+'-'+monthly_revenue['Month'].astype(str)

sns.lineplot(data=monthly_revenue[:10], x='month_year',
y='Total_Cost')
```

```
<Axes: xlabel='month_year', ylabel='Total_Cost'>
```



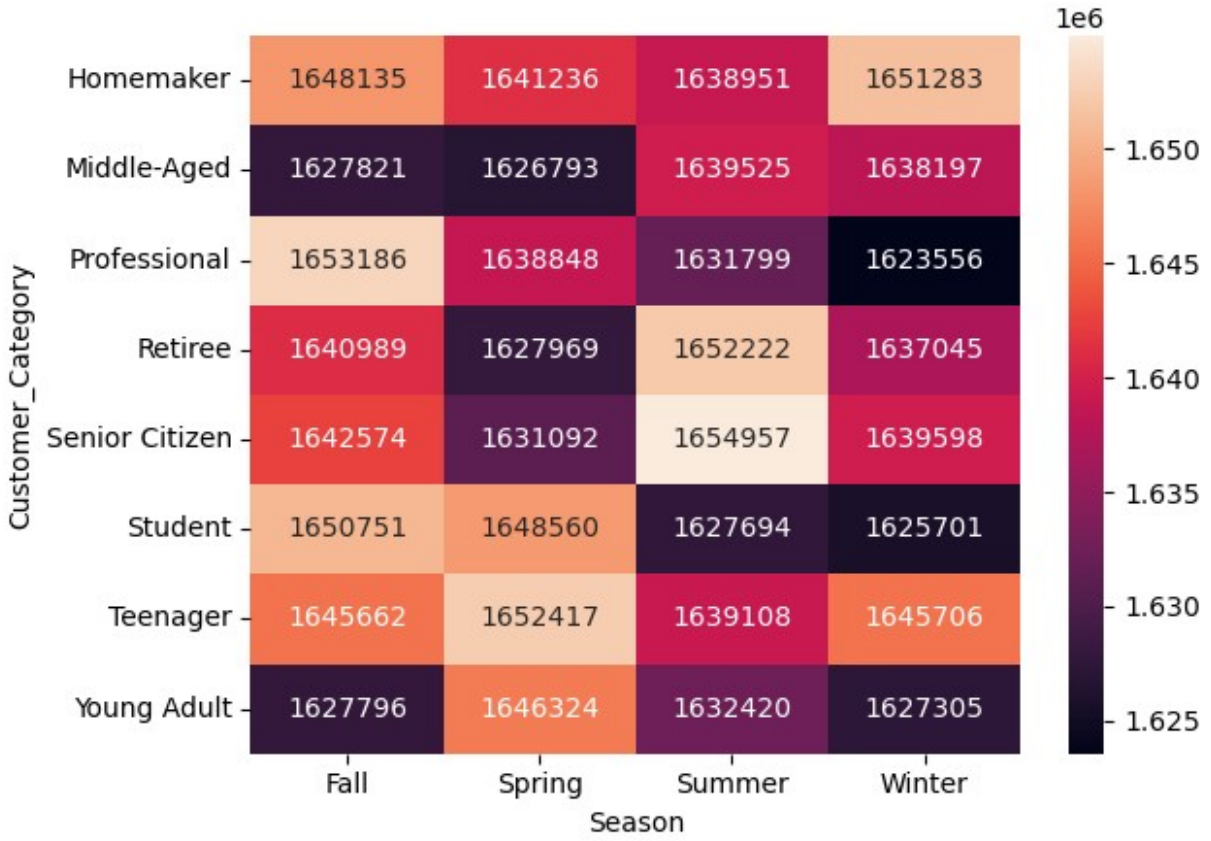

```
# 4.Heatmap or stacked bar showing revenue by season and customer
category
#we need to use pivot table to create a heatmap and Heatmaps can be
created using seaborn

pivot_data=df.pivot_table(values='Total_Cost',
index='Customer_Category', columns='Season', aggfunc='sum')
pivot_data
```

Season	Fall	Spring	Summer	Winter
Customer_Category				
Homemaker	1648135.38	1641236.04	1638951.37	1651283.18
Middle-Aged	1627821.10	1626793.49	1639525.01	1638197.44
Professional	1653185.88	1638847.64	1631799.05	1623556.20
Retiree	1640988.57	1627968.66	1652221.94	1637044.78
Senior Citizen	1642573.51	1631091.89	1654956.92	1639597.87
Student	1650751.04	1648559.84	1627693.87	1625701.41
Teenager	1645661.86	1652417.16	1639107.97	1645706.19
Young Adult	1627796.37	1646324.03	1632419.66	1627305.08

```
sns.heatmap(pivot_data, annot=True, fmt=".0f")
```

```
<Axes: xlabel='Season', ylabel='Customer_Category'>
```


1e6
Homemaker
Middle-Aged
Professional
Retiree
Senior Citizen
Student
Teenager
Young Adult
Customer_Category
1648135
1641236
1638951
1651283
1627821
1626793
1639525
1638197
1653186
1638848
1631799
1623556
1640989
1627969
1652222
1637045
1642574
1631092
1654957
1639598
1650751
1648560
1627694
1625701
1645662
1652417
1639108
1645706
1627796
1646324
1632420
1627305
Fall
Spring
Summer
Winter
Season
1.650
1.645
1.640
1.635
1.630
1.625